ActionComplete: Sample-Efficient Post-Training of VLAs via Residual RL

Saksham Sharma
saksham2
Robotics Institute
Carnegie Mellon University

Abhikhya Tripathy
abhikhyt
Robotics Institute
Carnegie Mellon University

Stephen Oduh
soduh
College of Engineering
Carnegie Mellon University

Abstract: Learning robotic manipulation policies that are both effective and sample-efficient remains a major challenge, particularly for tasks requiring precise physical interactions of the robot with the environment. Although end-to-end reinforcement learning (RL) has shown promise and been widely used for such tasks, it often suffers from slow convergence, sensitivity to reward design, and suboptimal exploration in complex environments. In this work, we investigate a framework that leverages pretrained base policies with rich prior knowledge, often learned via behavior cloning (BC), and adapts them to challenging manipulation tasks through the addition of a residual policy learned via online reinforcement learning. Instead of training from scratch, the residual policy only learns small actions to correct the base policy, resulting in more effective and sample-efficient learning. We specifically use Vision-Language-Action (VLA) models, which are pretrained on large amounts of data, as our base policy. We further highlight limitations of naive exploration and propose more informed exploration strategies beyond low-level action imitation.

1 Introduction

Robotic manipulation is a core problem in robotics, with applications ranging from industrial assembly to household assistance. Many manipulation tasks require precise control, contact-rich interactions, and robustness to uncertainty, making them challenging to solve efficiently with supervised learning methods. Reinforcement learning (RL) provides a general framework for learning manipulation policies through interaction and feedback from the environment. However, end-to-end RL methods are often slow to converge, sample-inefficient, and sensitive to reward design, which is done manually in most cases. Thus, in complex tasks where the agent lacks prior knowledge about the structure and dynamics of the environment, RL methods can fail. A promising alternative is to leverage pretrained large models that already encode rich prior knowledge from large datasets, such as Vision-Language-Action (VLA) models for robotics. These models perform well at predicting coarse robotic actions given visual input of the environment and language instructions, and can generalize across tasks and environments at this coarse level. However, due to limitations in how visual inputs are tokenized and processed, lack of interaction with the environment, and the frequency at which they can generate actions and get feedback from the environment, VLA models struggle with precise, closed-loop manipulation. Thus, VLA models behave well as high-level trajectory planners rather then accurate low-level controllers.

In this work, we aim to combine the strengths of pretrained models and reinforcement learning, by using a VLA model as a base policy and learning a residual reinforcement learning policy on top of it to predict corrective actions. The VLA model provides high-level action predictions using task-relevant priors, and the residual RL predicts small fine-grained actions to add to those predicted by the base policy. The residual RL policy is trained online through interactions with the environment, allowing it to learn precise adjustments that are beyond the scope of the base model. We keep the base VLA policy frozen throughout and train only the RL, leveraging the adaptability and explo-

ration ability of RL, and retaining the rich prior knowledge of the VLA model. Another advantage of keeping the base model frozen is that we don't need large amounts of data for fine-tuning the VLA model through SFT for different environments. The residual policy learns optimal behavior for each environment through interactions and feedback, achieving both sample efficiency and precise control. Crucially, this can enable usage of these specialized policies as expert demonstrators in a self-improving data flywheel [1]: their successful trajectories are aggregated to fine-tune the generalist VLA in a DAgger-like fashion, continuously enhancing the base model's capabilities.

2 Problem Statement

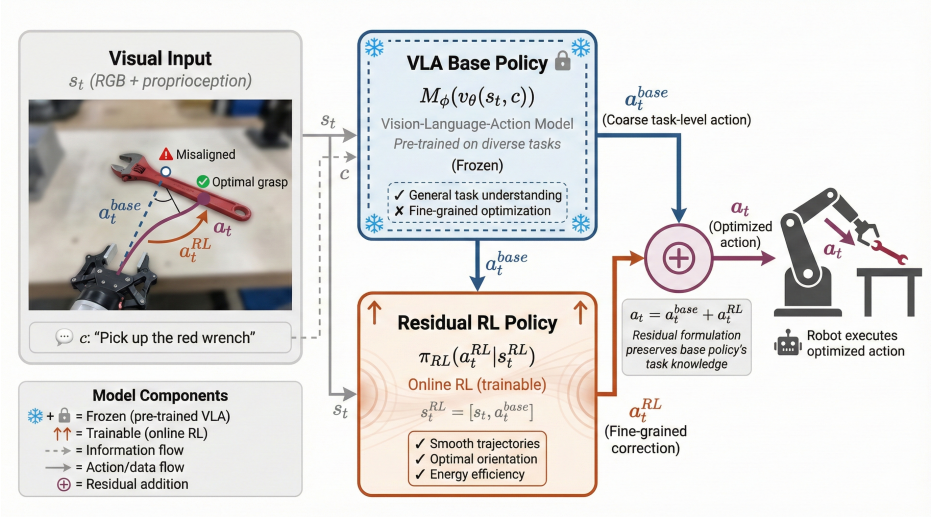

Figure 1: Overall Proposed Architecture

2.1 Residual Reinforcement Learning with Vision-Language-Action Model as Base Policy

Residual reinforcement learning aims to add small corrective actions on top of a pretrained base policy using an online reinforcement learning "residual" policy [2, 3].

In this work (as seen in Fig.1[1]), we choose our base policy to be a pretrained Vision-Language-Action (VLA) model which we keep frozen throughout. The model takes as input the visual (RGB) state of the environment through a specific camera angle, the robot proprioception information (position, velocity, orientation etc.), and a language instruction, and outputs an action a_t^{base} in a given timestep t.

We can formalize this as: $a_t^{base} = M_\phi(v_\theta(s_t, c))$, where v_θ is a vision-language model backbone parametrized by θ and M_ϕ is an action prediction head parametrized by ϕ, s_t is the environment state (visual input + robot proprioception information), and c is the language instruction.

We learn residual policy π_{RL} through online reinforcement learning. The residual policy's observation state is obtained by concatenating the base policy's observation state with its predicted action.

Thus, $s_t^{RL} = [s_t, a_t^{base}]$. In each timestep t, the residual RL policy predicts a corrective action $a_t^{RL} \sim \pi_{RL}(.|s_i^{RL})$, which is added to the base action.

This yields a combined policy:

$$\pi'(a_t|s_t, c) = M_\phi(v_\theta(s_t, c)) \cdot \pi_{RL}(a_t^{RL}|s_t^{RL}), \quad a_t = a_t^{base} + a_t^{RL} \tag{1}$$

In this work, we will implement 3 different reinforcement learning algorithms for the residual policy: On-policy Policy-gradient RL, Off-policy Q-function-based RL, Model-based RL.

[1] Generated using PaperBanana [4]

2.2 Environment

We use the *PegInsertionSide-v1* environment within ManiSkill robot simulator [5] with the Franka Emika Panda arm to demonstrate the performance of our algorithms. The goal task in this environment is to pick up an orange-white peg and insert the orange end into a box with a hole in one of its lateral faces. We choose this because it requires detailed perception and precise control.

2.3 Reward Function

We define the reward (per timestep) as a combination of stage-specific components, which are only non-zero when the action is in the corresponding stage. The stages are 1) grasping the peg at its tail (R_{grasp}), 2) orienting the peg axis to align with the hole's normal axis (R_{orient}), and 3) inserting the peg's head into the hole (R_{insert}). On successful insertion, a large reward $R_{success}$ is returned and other components are 0. The stages are identified through the relative positions and orientations of the peg and the hole. The full reward can be written as R where $\lambda_1, \lambda_2, \lambda_3 \in \{0, 1\}$ depending on the stage.

$$R = \lambda_1 R_{\text{grasp}} + \lambda_2 R_{\text{orient}} + \lambda_3 R_{\text{insert}} + R_{\text{success}}$$

$$R_{\text{grasp}} = -\alpha_1 \|p_{\text{gripper}} - p_{\text{peg-tail}}\| - \alpha_2 |\cos^{-1}(v_{\text{gripper}} \cdot v_{\text{peg}})| + \beta_1 I_{contact} - \beta_2 ||g_{width} - g_{optimal}||$$

$$R_{\text{orient}} = -\gamma_1 |\cos^{-1}(v_{\text{peg}} \cdot v_{\text{hole}})|$$

$$R_{\text{insert}} = -\delta_1 \|p_{\text{peg-head}} - p_{\text{hole}}\| + \delta_2 d_{\text{inserted}}$$

p_i denotes the position of the entity i, v_i the orientation of entity i, $I_{contact} = 1$ if gripper makes contact with peg tail and 0 otherwise, g_{width} is the width of the gripper and $g_{optimal}$ its optimal width for stably grasping the peg, and $d_{inserted}$ is the depth of insertion into the hole.
$\alpha_1, \alpha_2, \beta_1, \beta_2, \gamma_1, \delta_1, \delta_2$ are weighting hyperparameters.
When $d_{inserted} \geq d_{required}$, our task is successfully completed and $R_{success}$ is returned.

3 Proposed Modifications

3.1 Modification 1: Learning Exploration Strategies from Demonstrations

Once we have a combined policy π' that performs well on the standard peg insertion task, we would like to extend it to settings with additional environmental complexity. The key idea is that humans often solve complex tasks via trial-and-error (e.g., inserting a plug into a socket behind a door or under a table without seeing it, throwing a lasso around a distant pole, or disentangling a highly coiled wire). However, humans develop strategies to make their trial-and-error, or exploration, more efficient, and apply these to any similar scenario. Our goal is to teach the residual RL policy π_{RL} to learn such *strategies* for efficient exploration from very few demonstrations of the same strategies for complex tasks [6, 7]. Another benefit is that human strategies often involve inherently safe exploration behaviors, which can be crucial in safety-critical settings.

To concretize, consider peg insertion into a hole located behind an opaque door, on its hidden surface, where our policy cannot "see" it. A human would first establish contact between the peg head and the hidden surface, then explore it systematically in a sweeping manner. Learning this strategy guarantees finding the hole within a bounded time, unlike lifting and re-placing the peg for each failure. We want our policy to learn such high-level strategies for peg insertion, rather than low-level actions, from a small number of human demonstrations, so that it can expand to challenging setups like door-like occlusions. This design also allows incorporating additional sensory inputs, such as force or tactile feedback, into the residual policy without retraining the base policy with new input modalities. By using RL instead of behavioral cloning, the system moves beyond low-level action imitation to learn strategic exploration patterns, thus learning from sparse human demonstrations.

In this work, we implement a toy version of the above idea for the peg insertion task as a first step in this direction. We structure peg insertion as a contact-rich task whose main challenge is not low-level motor control but the sequencing of meaningful contact phases. We decompose the task into

five discrete phases - free-move, make-contact, sweep-surface, engage-hole, and insert - motivated by how a human or robot searches for a hole using surface contact rather than visual feedback. This strategy is also good for extension to the setup of hole behind an opaque door.

To implement this, we represent each demonstration as a sequence of DemoSegment objects, one per contact phase. For this toy case, we use a synthetic demonstration generator which produces five-phase demonstrations with a sweeping search. These demonstrations are used in two complementary ways. First, they supply a demo-conditioned sampler for initializing the RL policy's exploration distribution. The demonstrations are parsed into DemoSegment objects carrying continuous parameters such as sweep direction, step size, and approach velocity, which are pooled across all demonstrations to build a distribution over exploration behaviors. When the RL begins training, rather than exploring from a uniform or Gaussian noise distribution centered on zero, it samples its initial corrective actions from this demo-conditioned distribution, biasing early exploration toward sweep patterns and contact-search motions that were observed in demonstrations rather than random flailing unlikely to ever reach the hole. Second, each phase becomes a reusable skill in a learned hierarchy: a StrategyLearner builds a skill library from demonstration segments and estimates inter-skill transition probabilities, producing a learned mechanism graph that encodes the high-level exploration strategy. A HierarchicalQLearner then performs tabular Q-learning over abstract contact states - free, approached, surface-contact, hole-contact, engaged, and inserted - with the Q-table warm-started using an imitation bonus from demonstrations. Training uses epsilon-greedy exploration with reward shaping: a small per-step cost, positive rewards for reaching contact milestones, and a large terminal reward for successful insertion.

In this hierarchical setup, onee episode starts from a fresh peg-insertion scene and the policy repeatedly chooses one of the high-level skills. Each chosen skill is then executed by the low-level controller for several simulator steps, and the rewards from those low-level steps are added together. These are plotted for training steps in Fig.3. The episode ends when insertion succeeds, the environment terminates, or the maximum number of high-level skill choices is reached. This design operates at the skill level rather than the raw action level, so the policy learns when to sweep, when to probe for the hole, and when to commit to insertion - the strategic layer that naive RL exploration fails to discover within a practical sample budget.

3.2 Modification 2: VLM as a Learned Reward Signal

A core limitation of the residual RL framework, as well as end-to-end RL in general, is the reliance on hand-crafted reward functions. Designing such functions requires expert knowledge, is task-specific, and risks reward hacking where the agent optimizes for the measured proxy rather than the intended behavior. To address this, our first modification replaces the manually engineered dense reward with a *learned* visual reward model, Robometer [8], a scalable reward model trained on over 1 million robot trajectory comparisons using a Qwen3-VL vision-language backbone.

Robometer exposes two output heads: (1) a **progress head** that estimates the fraction of task completion $\hat{r}_{prog}(s_t, g) \in [0, 1]$ for a visual observation s_t and goal description g, and (2) a **preference head** that ranks pairs of trajectories. We use the progress score as a drop-in reward replacement:

$$r_t^{\text{rob}} = \hat{r}_{prog}(s_t,\, g) \tag{2}$$

where g is the language goal instruction passed to both the VLA and the reward model. Crucially, Robometer requires no task-specific engineering: the same model weights generalize across PickCube, PegInsertion, and other tasks by conditioning on the language goal alone.

In practice, Robometer is queried once per RL rollout step. Because the model runs inference at every environment step, we batch queries across the parallel environments and cache the visual encoding to reduce latency. The Robometer reward is normalized to have zero mean and unit variance over a running window to stabilize PPO training, as the raw progress score distribution varies across tasks.

4 Results

4.1 Performance of a Random Agent

Table 1: Average Reward per episode for PegInsertionSide-v1 with random actions

Rollout	Mean Reward
Rollout 0	0.017
Rollout 1	0.014
Rollout 2	0.013

From Table 1 we can see that random agent fails to consistently make progress toward successful insertion, highlighting the difficulty of the task and the necessity of structured learning and exploration strategies.

4.2 Baseline Comparison: On-policy Policy-Gradient vs. Off-policy Q-function-Based RL vs Model-based RL

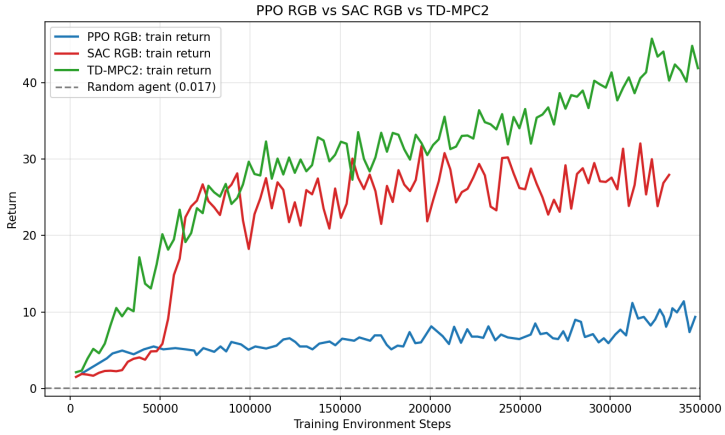

Figure 2: Mean Return vs. Training Environment Steps with PPO and SAC

To establish performance baselines for our residual RL framework prior to integrating the VLA base policy, we trained two standalone RL agents directly on the PegInsertionSide-v1 environment in ManiSkill. In adherence to the project requirements, we implement Proximal Policy Optimization (PPO - On-policy Policy-Gradient method), Soft Actor Critic (SAC - Off-policy Q-function-Based method), and TD-MPC2 (a model-based RL method with an implicit world model and model-predictive planning). All three agents take only RGB observations of the environment, and use the pd_joint_delta_pos control mode in ManiSkill.

4.3 Residual RL

The plots below (Fig.4 and 5) show the Returns vs. timesteps and Success vs. timesteps for the residual RL setup where base policy is RDT-1B [9] VLA and residual RL policy is SAC.

4.3.1 Analysis of Learning Curves

Both PPO and SAC obtain mean returns significantly higher than the random agent, demonstrating learning. SAC converges much faster than PPO, reaching a stable return of $\sim$25 compared to PPO's $\sim$6 at the same point. PPO's curve is smooth, reflecting variance reduction from generalized advantage estimation and reward normalization. However, its slope is very shallow, suggesting the agent struggles to extract precise spatial features from RGB frames alone. SAC's curve shows a sharp

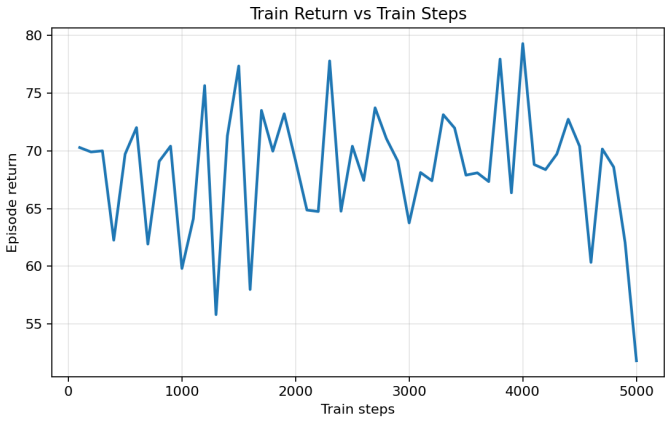

Figure 3: Return vs.Train Timesteps for Modification 1

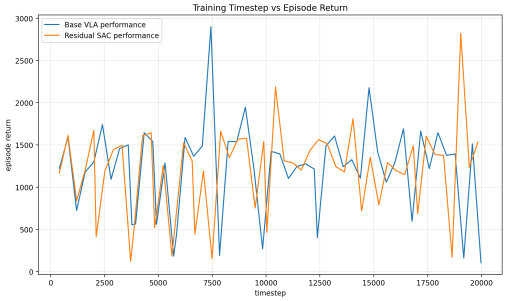

Figure 4: Return vs.Timesteps for VLA + SAC

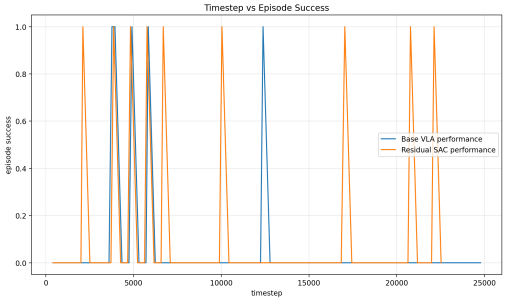

Figure 5: Success vs.Timesteps for VLA + SAC

increase around 30k–50k steps, corresponding to the replay buffer reaching sufficient coverage for reliable Q-function bootstrapping. TD-MPC2 leverages a world model learned from experience, which allows it to plan ahead and explore more efficiently than pure model-free approaches. It exhibits the fastest initial rise in return among the three methods, reaching high returns early by simulating rollouts internally. However, model errors in the learned world model can cause instability, visible as variance in training.

4.3.2 Analysis of Success Rates

All implementations log success_once metric indicating whether insertion was achieved in an episode or not. We observe that in all our experiments with PPO, SAC and TD-MPC2, the agents achieved a success rate of 0. We refer the reader to our website which shows videos of our trained agents performing the PegInsertion task in the ManiSkill environment. We see that all our agents have learned to pick up the peg and are performing decently on orienting it with respect to the hole's axis and bringing it close to the mouth of the hole, especially in comparison to the random agent. The agents fail at the most difficult and fine-grained phase of this manipulation task - insertion - which requires perfect alignment of peg and hole and their axis orientations. This results in our observed success rates of 0. Since, we see the mean returns still rising at the end of the plot, we believe that training for many more timesteps could lead to non-zero success rates. However, this training would be very time-consuming.

4.3.3 Analysis of Failure Cases

Our task's contact-rich nature results in millimeter-level misalignments preventing insertion. Hence, our RL-only agents without any prior regarding this task, and trained with just RGB inputs, require many timesteps to start performing reasonably well via exploration and would require significantly

more to start executing successful insertions. This is where our residual reinforcement learning setup would help. We propose to have a base VLA policy which has the required prior of predicting coarse robot actions conditioned on RGB observations, and train residual RL policies on top of it for fine-grained action corrections for achieving success. A major motivation behind this is that due to the pretrained knowledge of the base VLA, the agent's residual RL training would start at a much better stage than training RL from scratch. This would enable the RL policies to systematically explore and improve upon the fine-grained manipulation corrections without the total number of timesteps being too large. Hence, with our base VLA + residual RL setup we aim to achieve faster convergence to successful peg insertions.

4.4 Comparison of performance across all methods and analysis

We observe that among the RL-only algorithms, the model-based TD-MPC2 performs the best, the analysis for why this is the case has been detailed above. The residual RL implementations (VLA + PPO and VLA + SAC) perform significantly better than the RL-only algorithms, achieving successful insertions in some episodes as well, which wasn't the case earlier. This validates our claim for improved results from a coarse action predictor pretrained with large-scale data and priors, and a fine-grained RL-based controller for smaller corrections.

4.5 Comparison of performance across the two modifications and analysis

4.5.1 Modification 1: Learning Exploration Strategies from Demonstrations

Fig.3 shows the train return vs. timesteps for the modification 1, as described in the earlier section. For the toy implementation, we pass the state information. We see that the returns do not increase too much. The reason for this becomes clear when we refer to the output ManiSkill videos for this task using our modification 1 method (Please refer to our course project website for the same). We see that the robot gripper recognizes that it needs to perform the stage of peg gripping and proceeds to do so. However, it is unable to successfully finish that phase due to shortcomings of the low-level controller and thus, cannot proceed to the next stage. We also show videos where we force the robot to move to the next *learned* phase even if it isn't finishing a phase successfully. In these videos we see that it's trying to follow through with the sequence it has learned - grip, move to surface, make contact, sweep etc. This indicates that our setup is learning the high-level mechanism/strategy but isn't able to succeed with the full task, the major bottleneck being the effectiveness of the low-level fine-grained controllers.

4.5.2 Modification 2: VLM as a Learned Reward Signal

Having established that standalone RL agents fail to solve PegInsertionSide-v1 reliably, we explore the usage of residual RL on top of VLA. Additional we provide some experimental results on PickCube-v1 task as well. In the above experiments we pass only state information to the residual RL policy. We compare three conditions, all using PPO as the residual RL algorithm on top of the frozen RDT-1B [9] VLA:

- **VLA Baseline**: the VLA policy is evaluated without any active residual correction (action scale fixed at 0). This measures the VLA's out-of-the-box performance as an upper-bound reference.
- **VLA + ManiSkill Reward**: Residual RL trained with the dense task reward from ManiSkills3 simulator (according to the reward function defined before).
- **VLA + Robometer (Modification 2)**: Residual RL trained with Robometer progress scores as the reward signal, with an adaptive action scale that grows from 0 to 0.1 over training.

For PegInsertion (Fig 6), residual RL is able to eventually move above the VLA-only baseline, but the improvement is small and unstable. Plain residual RL starts by hurting performance, which likely

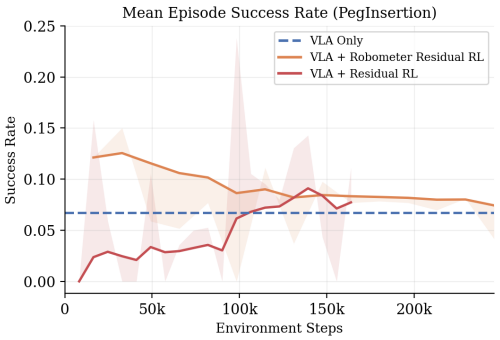

Figure 6: Success vs.steps for Peg Insertion Env

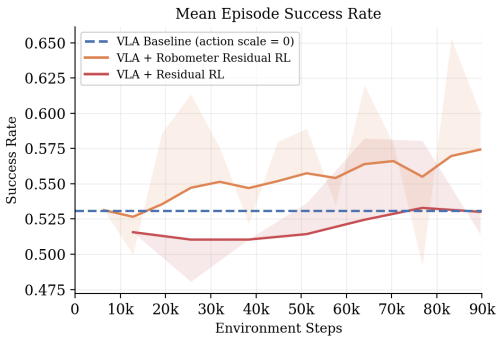

Figure 7: Success Rate vs.steps for Pick Cube

means early residual actions perturb the VLA's already reasonable trajectory. After more training it begins to learn useful corrective actions, but peg insertion remains highly sensitive to small pose and contact errors. Robometer residual RL starts higher, suggesting the learned reward gives better early guidance, but it gradually declines because visual progress is not precise enough to distinguish "near the hole" from a truly insertable pose.

For PickCube (Fig 7), Robometer is much more effective. The VLA baseline is already strong, so plain residual RL mostly learns to recover back to baseline rather than clearly improve it. Robometer residual RL, however, steadily rises above the baseline, showing that semantic progress rewards help when task progress is visually clear and well aligned with success. Overall, Robometer helps more on PickCube than PegInsertion because cube picking has smoother, visually observable stages, while peg insertion is dominated by fine contact precision.

These results suggest that Robometer reward shaping is most effective when task progress is visually observable and smoothly correlated with success. On PickCube, this produces a consistent improvement over the VLA baseline, whereas on PegInsertion the reward model improves early coarse progress but cannot reliably distinguish near-contact failures from successful insertion states.

4.6 Conclusions and Future Work

RL-only policies often struggle to achieve good returns and, more importantly, achieve successful completions on complex fine-grained tasks. Thus, if we let an end-to-end RL-only policy train for such tasks, it will take a very long time to show successful learning, or might never learn. To expedite the learning convergence, we implement and experiment with residual RL - the base VLA provides coarse actions based on its large-scale learned priors, and the residual RL policy learns online through dense environment rewards to perform fine-grained actions. We also implement a toy experiment for making policies learn mechanisms or strategies rather than low-level actions, which would be more generalizable. We notice that a very big limitation for each method being the inefficacy of low-level fine-grained actions, which tarnish the performance of our modifications as well. Our major future work is to improve the hierarchal setting to involve gradient flow from low level policy to high level strategy to allow more nuanced actions to be conducted, like picking an egg.

4.7 Project Website

Videos and visualizations of agent behavior in the environment are available on our project website: https://osobotu.github.io/rlproject/.

References

[1] W. Xiao, H. Lin, A. Peng, H. Xue, T. He, Y. Xie, F. Hu, J. Wu, Z. Luo, L. Fan, et al. Self-improving vision-language-action models with data generation via residual rl. *arXiv preprint arXiv:2511.00091*, 2025.

[2] L. Ankile, A. Simeonov, I. Shenfeld, M. Torne, and P. Agrawal. From imitation to refinement-residual rl for precise assembly. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 01–08. IEEE, 2025.

[3] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine. Residual reinforcement learning for robot control. In *2019 international conference on robotics and automation (ICRA)*, pages 6023–6029. IEEE, 2019.

[4] D. Zhu, R. Meng, Y. Song, X. Wei, S. Li, T. Pfister, and J. Yoon. Paperbanana: Automating academic illustration for ai scientists. *arXiv preprint arXiv:2601.23265*, 2026.

[5] S. Tao, F. Xiang, A. Shukla, Y. Qin, X. Hinrichsen, X. Yuan, C. Bao, X. Lin, Y. Liu, T. kai Chan, Y. Gao, X. Li, T. Mu, N. Xiao, A. Gurha, V. N. Rajesh, Y. W. Choi, Y.-R. Chen, Z. Huang, R. Calandra, R. Chen, S. Luo, and H. Su. Maniskill3: Gpu parallelized robotics simulation and rendering for generalizable embodied ai. *Robotics: Science and Systems*, 2025.

[6] M. Vecerik, T. Hester, J. Scholz, F. Wang, O. Pietquin, B. Piot, N. Heess, T. Rothörl, T. Lampe, and M. Riedmiller. Leveraging demonstrations for deep reinforcement learning on robotics problems with sparse rewards, 2018. URL https://arxiv.org/abs/1707.08817.

[7] Y. Shi, Z. Chen, Y. Wu, D. Henkel, S. Riedel, H. Liu, Q. Feng, and J. Zhang. Combining learning from demonstration with learning by exploration to facilitate contact-rich tasks, 2021. URL https://arxiv.org/abs/2103.05904.

[8] A. Liang, Y. Korkmaz, J. Zhang, M. Hwang, A. Anwar, S. Kaushik, A. Shah, A. S. Huang, L. Zettlemoyer, D. Fox, et al. Robometer: Scaling general-purpose robotic reward models via trajectory comparisons. *arXiv preprint arXiv:2603.02115*, 2026.

[9] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu. Rdt-1b: a diffusion foundation model for bimanual manipulation. *arXiv preprint arXiv:2410.07864*, 2024.